

Documentation for Generic Test Case Generation Template

This document outlines the structure and guidelines for using the "GENERIC TEST CASE GENERATION TEMPLATE" to generate test cases in various formats.

1. Introduction

This template introduces a standardized process for creating **detailed, categorized test cases** using minimal input data. The test cases will first be delivered in **CSV format** for easy review, management, and usage across various environments.

➔ JSON format generation is optional and only triggered upon user request.

2. Template Breakdown

The template is divided into the following sections:

2.1. INPUT SECTION

This section defines the contextual inputs required for generating the test cases:

Line of Business: [Enter Line of Business]

Epic: [Enter Epic Name or ID]

Feature: [Enter Feature Name or ID]

User Story Details:

- User Story ID: [Enter ID]
- Title: [Enter Title]
- Description: [Enter Description]
- Acceptance Criteria: [Enter Acceptance Criteria]

Environment & Application:

- Environment: [Select from: Portal | Mobile | POS]
- Application Types: [Enter types, e.g., Native App, Web App, etc.]

2.2. TEST CASE GENERATION PROMPT

Instruct the AI to:

- Generate minimum 20 test cases (can exceed based on category needs)
- Cover all testing categories
- Save the output as a downloadable .csv file
- Do not generate JSON unless requested

2.3. REQUIRED TEST CATEGORIES (Minimum Distribution):

Distribute test cases to meet or exceed the following thresholds:

1. FUNCTIONAL VALIDATION

Happy Path: Testing the primary, successful scenario.

Negative Scenarios: Testing with invalid or unexpected inputs.

Boundary Conditions: Testing at the limits of valid inputs.

2. DATA VALIDATION

Input Fields: Testing the validation rules for different input fields.

Data Types: Testing with various data types (e.g., integers, strings, Booleans).

3. ERROR HANDLING

System Errors: Testing how the system handles internal errors or exceptions.

Validation Errors: Testing the display and handling of user input validation errors.

4. INTEGRATION TESTING

System Interactions: Testing the interaction between different components or systems.

Data Flow: Testing the correct flow and transformation of data between modules.

5. SECURITY/PERMISSIONS

Access Control: Testing user roles and permissions to ensure authorized access.

6. DATABASE

Data Persistence: Testing if data is correctly saved and retrieved from the database.

Query Validation: Testing the validity and efficiency of database queries (if applicable).

7. PERFORMANCE

Response Time: Testing the system's responsiveness under normal load.

Load Handling: Testing how the system behaves under increased user load.

8. NETWORK

Connectivity: Testing the system's ability to connect to necessary network resources.

Latency: Testing the system's behavior under different network latency conditions.

9. UI/UX

Usability, Visuals, Accessibility

2.4. OUTPUT FORMATS

This section defines the structure for each of the required output formats.

2.5. VERIFICATION CHECKLIST

This checklist helps ensure that the generated test cases meet the specified requirements.

2.6. NOTES

This section provides additional guidelines for the AI model.

3. Usage Instructions

To use the **Generic Test Case Generation Template** effectively, follow the steps below:

Step 1: Fill in the Input Section

Complete the fields in the [INPUT SECTION] to provide contextual details for the test case generation process:

- Line of Business: Identify the business domain or vertical.
- Epic: Specify the Epic name or ID associated with the feature.
- Feature: Mention the relevant Feature name or ID.
- User Story Details: Provide the ID, Title, Description, and Acceptance Criteria.
- Environment & Application Details: Select the environment (Portal, Mobile, POS) and specify applicable application types.

Step 2: Use the Prompt for Generation

Once the input section is filled out:

- Pass the complete input and prompt to the AI model.
- Ensure the model generates a minimum of 20 CSV-based test cases, with balanced coverage across all the specified test categories.

Step 3: Review the CSV Output

The output should be a **properly formatted CSV file** with the required headers

Step 4: Optional JSON Generation

After validating the CSV file, you may request the same test cases in **Simplified JSON format** to upload them to Azure DevOps.

This is the **full prompt**. You can edit it and add more data to help generate additional scenarios.

===== INPUT SECTION =====

Line of Business: [Enter the Line of Business]

Epic: [Enter Epic Name/ID]

Feature: [Enter Feature Name/ID]

User Story Details:

- User Story ID: [Enter ID]
- Title: [Enter Title]
- Description: [Enter Description]
- Acceptance Criteria: [Enter Acceptance Criteria]

Environment & Application Details:

- Environment: [Choose from Portal, Mobile, POS]
- Application Types: [Enter the types of applications applicable]

===== TEST CASE GENERATION PROMPT =====

Generate a ****minimum of 20 detailed test cases**** in the form of a ****CSV file****. The test cases must cover all major testing categories listed below and may exceed 20 if necessary for full and balanced coverage.

📌 Save the test cases into a downloadable `.csv` file. Ensure that the file is saved correctly so that the download link works. Use a link (e.g., [Download CSV File](sandbox:/mnt/data/TestCases.csv)) that allows the user to download the CSV file directly.

Do ****not**** generate the JSON format unless the user explicitly requests it after reviewing the CSV.

===== REQUIRED TEST CATEGORIES =====

Ensure balanced distribution across these categories:

- FUNCTIONAL VALIDATION (at least 6)

- Happy Path
- Negative Scenarios
- Boundary Conditions

- DATA VALIDATION (at least 4)

- Input Fields
- Data Types

- ERROR HANDLING (at least 4)

- System Errors
- Validation Errors

- INTEGRATION TESTING (at least 4)

- System Interactions
- Data Flow

- SECURITY/PERMISSIONS (at least 2)

- Access Control

- DATABASE (at least 2)

- Data Persistence
- Query Validation

- PERFORMANCE (at least 2)

- Response Time
- Load Handling

- NETWORK (at least 2)

- Connectivity
- Latency

- UI/UX (as applicable)

- Usability
- Visual Consistency
- Accessibility

===== STEP-BY-STEP PROCESS =====

1. Generate test cases ****only in CSV format**** first. Use the following headers exactly:

Generate test cases in a step-based table with headers written only in the first line:

ID, Work Item Type, Title, Test Step, Step Action, Step Expected

- Each test case is separated by a separate blank line.
- "Work Item Type" is "Test Case" and appears once per test case in the first row of each case.
- "ID" column is set in per testcase.
- Subsequent rows for the same test case contain steps (Test Step, Step Action, Step Expected).
- Leave first row empty for columns (Test Step, Step Action and Step Expected)
- Convert the final output to CSV format.

2. Provide a downloadable CSV file link so that the user can directly download and review the file. For example, use a link similar to: [Download CSV File](sandbox:/mnt/data/TestCases.csv).

3. After the user validates the CSV file, ask:

_ "Would you like to generate the same test cases in Simplified JSON format?" _

4. If the user agrees, then generate the JSON using the following structure precisely. IMPORTANT: In the JSON steps, for each <parameterizedString isformatted='true'> tag, do not include the characters ";" or ":" anywhere in the text strings.

```
``json
[
  {
    "title": "Verify [Short Summary of Test Scenario]",
    "steps": "<steps id='0' last='<last_step_id>'><step id='1' type='ActionStep'><parameterizedString isformatted='true'>Instruction using only periods or commas as separators</parameterizedString><parameterizedString isformatted='true'>Expected result using only periods or commas as separators</parameterizedString></step><step id='2' type='ActionStep'><parameterizedString isformatted='true'>Additional instruction using only periods or commas as separators</parameterizedString><parameterizedString isformatted='true'>Additional expected result using only periods or commas as separators</parameterizedString></step></steps>",
    "url": "<Link to related user story or work item (e.g., Azure DevOps URL)>",
    "tags": "AI Generated"
```

```
}  
]  
...
```

Example of the JSON steps structure:

```
<steps id='0' last='3'>  
  <step id='1' type='ActionStep'>  
    <parameterizedString isformatted='true'>Step instruction. Launch the application using the  
provided credentials</parameterizedString>  
    <parameterizedString isformatted='true'>Expected result. Application loads successfully and  
displays the homepage</parameterizedString>  
  </step>  
  <step id='2' type='ActionStep'>  
    <parameterizedString isformatted='true'>Step instruction. Perform login using valid credentials and  
wait for the dashboard screen</parameterizedString>  
    <parameterizedString isformatted='true'>Expected result. The system verifies the credentials and  
navigates to the dashboard</parameterizedString>  
  </step>  
  <step id='3' type='ActionStep'>  
    <parameterizedString isformatted='true'>Step instruction. Validate the display of user settings  
after login</parameterizedString>  
    <parameterizedString isformatted='true'>Expected result. User settings are displayed correctly  
with appropriate details</parameterizedString>  
  </step>  
</steps>
```

===== VERIFICATION CHECKLIST =====

- ✓ Minimum 20 test cases
- ✓ All required testing categories are represented
- ✓ CSV file is generated first
- ✓ JSON format is generated only on user confirmation
- ✓ Test case content is consistent across both CSV and JSON formats

- ✓ File structure is valid and consistent

===== NOTES =====

- Do not proceed with JSON format unless explicitly requested by the user.
- Ensure the CSV test cases are professional, structured, and clearly labeled.
- Use a placeholder URL if none is provided (e.g., [https://dev.azure.com/organization/project/_workitems/edit/\[ID\]](https://dev.azure.com/organization/project/_workitems/edit/[ID])).
- Maintain consistency in step descriptions and expected outcomes.
- In the JSON steps section, do not include the characters ";" or ":" or "&" in any text within the `<parameterizedString isformatted='true'>` tags. Use alternative punctuation such as periods or commas.

Using Postman Collection to Create Multiple Test Cases

Follow these step-by-step instructions to execute the generated test cases using the Postman Collection:

✓ Step 1: Import the Collection

Open Postman.

Click on "Import" and select the provided Postman Collection JSON file.

✓ Step 2: Locate the Request

Inside the imported collection, open the request titled: **"Create Multiple Test Cases"**.

✓ Step 3: Configure Azure Variables

Navigate to the Collection tab named [Azure].

Update the following collection-level variables:

- organization – Your Azure DevOps organization name.
- project – Your Azure DevOps project name.
- token – Your Azure personal access token.
- workItemId – The target Work Item ID (e.g., user story ID) to associate the test cases with.

✓ Step 4: Launch the Collection Runner

- Go back to the Azure Collection.
- Click on the three-dot icon (:) next to the collection name.
- Select "Run" from the dropdown menu.
- A new Collection Runner window will open.

✓ Step 5: Upload the JSON File

In the Collection Runner:

- Click on "Select File" under the Data section.
- Choose the JSON test case file generated by the AI model.

 Step 6: Run the Collection

- Click on "Run" to begin executing the requests.